HybridKernel: A Pre-Registered Profiler Gate for Hybrid Attention/SSM Boundaries

Anonymous authors
Paper under double-blind review

Abstract

Hybrid attention/state-space language models invite a systems question: do attention-to-SSM layer boundaries create avoidable conversion, materialization, launch, or locality overhead that a fused boundary operator could remove? We report a Mac-feasible pre-GPU gate for this question, not a speed result. Architecture maps show large boundary-crossing activation streams, but a runtime audit weakens broad novelty because current vLLM hybrid SSM support already handles important state-layout and transfer paths; a kernel contribution exists only if native traces reveal overhead not already handled by that machinery. A threshold model says Granite 4.0 H needs roughly 25% genuinely avoidable boundary traffic at 60% recovery to clear a 3% proxy gain; Qwen3-Next needs 10.4% but is less directly matched to the Granite Mamba2 hypothesis. The artifact is a source-line audit, control feasibility matrix, fixed-request vLLM driver, native packet skeleton generator, pre-registered profiler-analysis parser, artifact-completeness verifier, and Mac-local Triton interpreter correctness for a toy boundary primitive. No throughput, latency, HBM, or GPU speedup claim is made.

1 Hypothesis

HybridKernel tests whether adjacent attention and SSM layers expose boundary-local overhead not already removed by serving systems. Candidate costs include layout conversion, residual-stream materialization, host launch gaps, cache-locality loss, or redundant memory traffic. The hypothesis is deliberately narrow: a useful kernel must compose with existing vLLM hybrid mechanisms rather than replace them.

2 Current Evidence

Artifact	Result	Decision
Architecture map	Granite has 8 boundaries and 20.0% boundary stream fraction; Qwen3-Next has 23 inferred boundaries and 47.9%.	Inspectable, not speed evidence.
Runtime audit	vLLM already handles hybrid state layout and transfer paths.	Broad novelty weakened.
Threshold model	Granite needs about 25.0% avoidable boundary traffic at 60% recovery for a 3% proxy gain.	Mac kernel work not justified.
Profiler driver	Fixed-request OpenAI-compatible driver dry-runs locally; its optional profile bracket calls vLLM `/start_profile` and `/stop_profile` around replay for dynamic Nsight capture. The runbook profiles the vLLM server, not just the HTTP client.	Native gate is less likely to be mis-run.
Analysis parser	JSON profiler summaries are reduced into avoidable-boundary share, recoverable-gain upper bound, median/IQR, and bootstrap intervals over repeated reduced rows. Rows must explicitly record matched control time, recoverable fraction and basis, dtype, CUDA-graph state, batch/request shape, control segment, row role, artifact hashes, reduction command, and distinct run ID.	Decision rule is pre-registered and incomplete, unhashable, or duplicated rows cannot inflate gains.
Artifact verifier	Native run directories are checked for metadata, server-side Nsight Systems and Compute scope, explicit NCU launch selection, row-level reduction input manifest, Nsight artifacts, separate Nsight server profiler logs, non-dry-run client replay JSON, readout rows, distinct repeated metric rows for the same model/config, and matching profiler-analysis outputs.	Client-only, warmup-only, dry-run, failed-request, duplicated-trace, stale-analysis, mixed-config repeat, manually selected NCU launches without provenance, unmanifested reduction windows, and incomplete-log evidence is rejected.
Packet generator	A local script creates the required native run-packet skeleton with sentinel markers.	GPU operators get a consistent handoff; skeletons are rejected as evidence until filled.
Synthetic packet fixture	A Mac-local fixture documents the packet shape and exercises the checker with placeholder Nsight files.	Fixture is not profiler evidence.
Packet checklist	The native handoff now has a single required packet layout and stop-local-work decision.	Local readiness is saturated until GPU data exists.
Source/control audit	Source-line audit table and control feasibility matrix record what public surfaces were checked and which native controls are still placeholders.	Review hardening only; no proof of absence.
Triton correctness	The toy boundary primitive passes Phase 4 interpreter tests under the repo-local `triton-cpu` source install across block tails, fp16, non-contiguous inputs, and shape errors. The owned Mac suite passes with one opt-in CPU-backend test skipped by default; the opt-in non-interpreter CPU-backend check also passes when Homebrew GCC library paths are made explicit.	Kernel logic only; not CUDA or speed evidence.

Table 1: HybridKernel evidence before native GPU profiling.

3 Profiler Gate

The next experiment is a native NVIDIA/vLLM run with Nsight Systems and Nsight Compute. It must show a distinct boundary-local overhead attributable to attention/SSM transitions, not warmup, graph capture, batching, graph-state changes, dtype changes, or unrelated kernels. The runbook now specifies exactly how to reduce Nsight traces into parser fields: total step time, boundary-local time, matched non-boundary control time, recoverable fraction, dtype, CUDA-graph state, batch/request shape, and control segment. Promotion requires at least three repeated runs for the same model/config whose recoverable-gain upper bound clears 3%. This first full-matrix pass would promote only the boundary-overhead investigation and prototype-kernel plan; it would still not authorize a paper-level throughput or vLLM speedup claim without later end-to-end implementation evidence. The native control matrix is fixed before profiling: Granite 4.0 H boundary rows are the

primary rows, same-model Granite non-boundary rows are the same-family controls, and Qwen3-Next or another preregistered feasible hybrid row is the cross-family falsification control. The checker rejects metric rows that fall outside the registered model role, control family, control segment, boundary direction, dtype, CUDA-graph state, batch shape, prompt/decode/request-token shape, and request status matrix. The analysis parser computes

$$\text{gain}_{ub} = \frac{\max(\text{boundary ms} - \text{matched control ms}, 0)}{\text{total step ms}} \times \text{recoverable fraction}.$$

If repeated native summaries show less than 1% recoverable gain, the branch should be killed or shelved. Before any run is cited, the artifact verifier must also pass on the exact run directory, confirming environment metadata, server-side Nsight Systems and Nsight Compute scope, profiler artifacts, separate Nsight server profiler logs, non-dry-run client replay JSON with all request statuses marked `ok` and exact prompt/decode/request token counts, the pre-registered readout questions, at least three distinct repeated metric rows for the same model/config, distinct Nsight artifacts and time windows for those repeats, explicit `ncu_launch_selection` provenance tying the NCU kernel regex and launch skip/count to the source Nsight Systems artifact/window, a row-level `reduction_input_manifest.json` tying each metric row to source artifacts, SHA-256 digests, source time windows, reducer command, reducer script digest, and notes, three matched same-family controls and three cross-family falsification rows on the same request/runtime shape, matching client replay logs for every model named in the metric rows, and profiler-analysis outputs that match the same metric file. The analysis parser, not the artifact checker, decides whether primary rows promote, weaken, or kill the branch relative to the 3% promotion and 1% shelf thresholds. A packet generator creates this directory shape for GPU operators, but the checker rejects skeleton sentinels until real metadata, metrics, manifests, and readout entries replace them. This is an admissibility check only; it does not promote the method. Each reduced metric row must cite relative in-packet Nsight artifact paths with valid extensions and matching SHA-256 digests, so missing or external trace references are rejected, as are hash-mismatched references. If Nsight Systems shows no suspicious boundary kernel to target, the explicit `no_boundary_signal_kill` packet mode permits a reviewable negative run without fabricating an Nsight Compute target. This mode is accepted only when the packet is a clean kill below the 1% recoverable-gain shelf threshold and its readout/reduction notes explicitly record that no boundary-local Nsight Systems signal was found.

4 Claim Boundary

Allowed now: HybridKernel is a weakly alive profiler-driven systems branch with a runnable native measurement plan, source/control audit artifacts, and Mac-local interpreter correctness for one toy boundary primitive. Not allowed: throughput, latency, HBM, energy, occupancy, or vLLM speedup claims. The toy primitive is intentionally insufficient as the proposed systems kernel; it only checks indexing and Triton plumbing. If native profiling does not reveal separable boundary overhead, the branch should be killed.

5 Limitations

No native NVIDIA/vLLM profile has been run for this draft. The architecture map is an upper-bound traffic screen, the synthetic packet fixture only tests artifact admissibility, and the toy Triton primitive is not a serving kernel. The Mac-only implementation lane is closed until native profiling exists. The paper is therefore a measurement protocol and pre-GPU handoff, not evidence that a fused boundary operator improves real workloads. The native promotion checklist now runs the artifact checker with `--require-full-matrix`; a primary-only profile can inform a kill decision, but it cannot promote the branch without same-family control and cross-family falsification rows.

6 Artifacts

- experimental/hybridkernel/phase2/nvidia_vllm_profiler_runbook.md
- experimental/hybridkernel/phase2/profiler_driver.py
- experimental/hybridkernel/phase2/create_native_run_packet.py
- experimental/hybridkernel/phase2/analyze_profiler_metrics.py
- experimental/hybridkernel/phase2/check_profiler_run_artifacts.py
- experimental/hybridkernel/phase2/native_run_packet_checklist.md
- experimental/hybridkernel/phase2/native_control_matrix.json
- experimental/hybridkernel/phase2/cross_family_control_replacement_template.json
- experimental/hybridkernel/phase2/reduction_worksheet_template.tsv
- experimental/hybridkernel/phase1/source_line_audit_table.md
- experimental/hybridkernel/phase2/control_feasibility_matrix.md
- experimental/hybridkernel/phase2/mac_reproducibility_command.md
- experimental/hybridkernel/phase2/profiler_analysis_gate.md
- experimental/hybridkernel/phase2/pre_gpu_threshold_model.md
- experimental/triton_cpu_source_install_20260506.md
- experimental/hybridkernel/paper/reviewer_pack.md